

Designed by Remus Teodorescu 2026

LECTURE 7.1

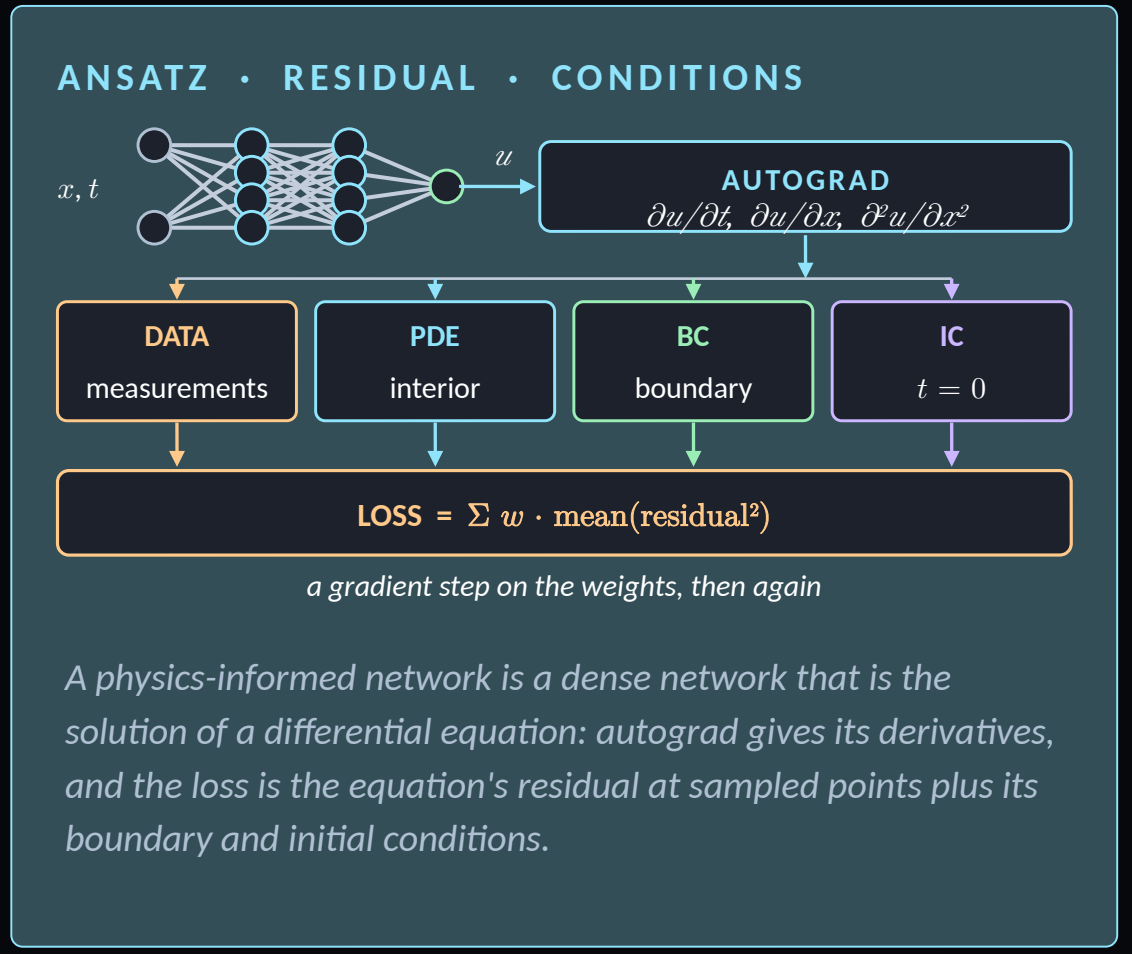
Physics-Informed Neural Networks

Remus Teodorescu

With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026



A physics-informed network is a dense network that is the solution of a differential equation: autograd gives its derivatives, and the loss is the equation's residual at sampled points plus its boundary and initial conditions.

Learning Objectives

SECTIONS, IN ORDER

1. **PINN** — what it is, the solution ansatz, smooth activations
2. **Formulation** — the operator form, boundary and initial conditions, forward and inverse
3. **Loss** — the composite loss, soft and hard conditions, weights, collocation

Autograd and the optimisers, Adam then L-BFGS, are L6.1's: this lecture uses them without teaching them again.

READ, AND WHAT EACH ANSWERS

Liu ch. 2.1	what is a PINN, and what can it represent?
Liu ch. 2.2	which equation, conditions and unknowns?
Liu ch. 2.2 – 2.9	how does each condition enter the loss?

Recap of L6.2 Training Philosophies

1 – 2 · PRE-TRAINING AND TRANSFER

1 What is transfer learning, and why does it work?

Starting from weights trained on a bigger, related task instead of from random ones. The first layers of a trained network hold general features that carry over, so they are kept, and only a new head is trained for your task. *UDL ch. 9.3.6; Ex_06 notebook 03*

2 How is a transferred network fine-tuned, and when does transfer stop helping?

Train the new head first, then unfreeze the last blocks at about a tenth of the learning rate, so that what the network knew is not overwritten. Transfer wins when labels are few; training from scratch catches up as the labels grow. *UDL ch. 9.3.6 – 9.3.7; Ex_06 notebook 03*

3 – 4 · REINFORCEMENT TRAINING

3 What kind of problem is reinforcement learning, and what does a policy compute?

Learning to act from a scalar reward that arrives late. The policy maps the state to an action and is trained to maximise the return, the sum of future rewards discounted by γ. In control terms: state, control input, controller and cost. *UDL ch. 19.1 – 19.2*

4 How does reinforcement learning differ from supervised learning, and when is imitating an expert the better choice?

There is no target for each input, only a reward after many actions, so training is noisy, needs many episodes and gives no guarantee. When an expert such as an MPC already exists, imitating it is supervised learning and far cheaper. *UDL ch. 19.5 – 19.7*

Physics-Informed Network

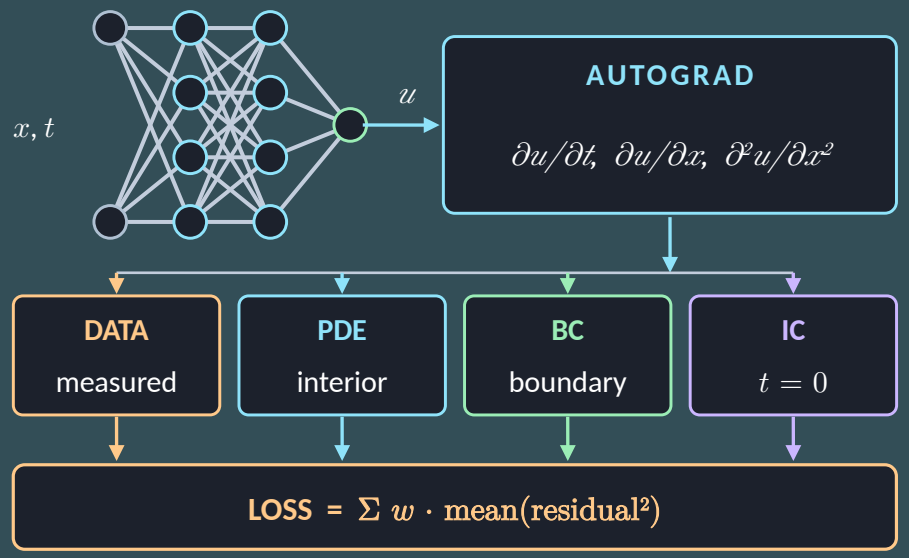
Physics-informed network — a neural network trained to be the solution of a differential equation: it takes the coordinates x and t and outputs the field u there. Its loss is the equation itself: autograd gives the derivatives of u, and the residual of the equation, squared and averaged over sampled points, is minimised together with the boundary and initial conditions.

AGAINST A MESH-BASED SOLVER

	FEM, FD	PINN
the unknowns	nodal values	the network's weights
the derivatives	a stencil or an element	autograd, exact
the geometry	meshed	sampled

TALK · 2 MIN How would you explain what a physics-informed network is, in your own words? Then, in the figure on the right, where do the equation and its conditions enter, and what does the data term add?

ONE NETWORK, ONE LOSS



a gradient step on the weights, then again

No labels are needed: the equation and its conditions are the targets.

Solution Ansatz

Solution ansatz — the network written as the approximate solution, $u(x,\, t) \approx \mathcal{N}(x,\, t;\, \theta)$. A wide enough network can come as close as needed to any continuous solution; training chooses θ, and how many points it needs is the Collocation slide's question.

THE NETWORK IS THE SOLUTION

$$u(\mathbf{x}, t) = \mathcal{N}_\theta(\mathbf{x}, t)$$

$\mathcal{N}_\theta$ the network, θ its parameters, u the field it stands for

- **one network** — u at any point, no interpolation
- **no mesh** — the weights are the unknowns

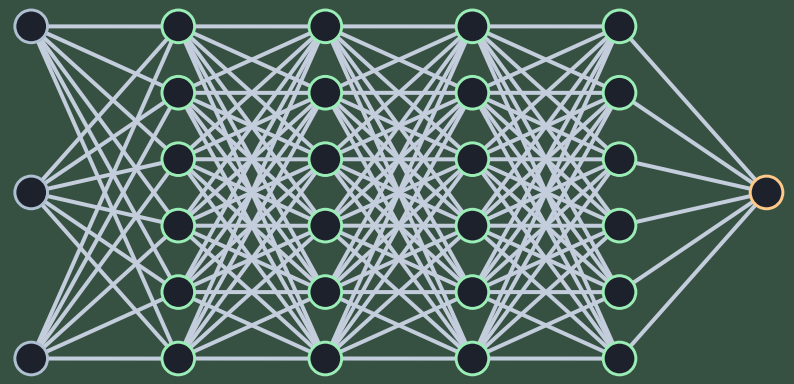
A MESH, AND A NETWORK



a mesh: values at its nodes

a network: a value at any point

EX_07.1: FOUR LAYERS OF 32



inputs	x, y and the current I
hidden	4 layers of 32 neurons, tanh
output	u, the field at that point

3 329 weights, shared by every point: they are what training solves for.

Smooth Activation Functions

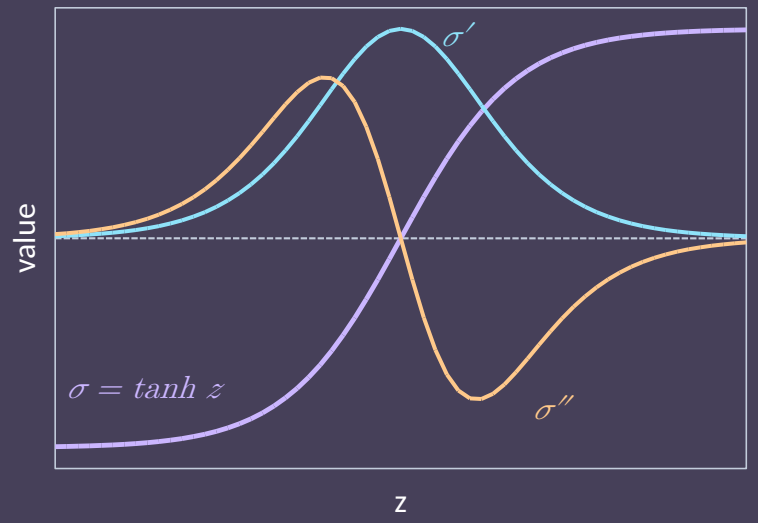
Smooth activation functions — without one, the layers collapse into one linear function, $u = \mathbf{a}\cdot\mathbf{x} + b$: a flat plane with $\nabla^2 u = 0$, which cannot bend into a hot spot or a wave. The activation gives the network curvature; a residual with k-th derivatives needs its k-th derivative to exist and not be zero. tanh is smooth to every order.

TANH AND WHAT THE RESIDUAL USES

	of tanh	the residual uses it for
$\sigma(z)$	$\tanh z$	the field u itself
$\sigma'(z)$	$1 - \tanh^2 z$	first order: u_t, u_x
$\sigma''(z)$	$-2 \tanh z\,(1 - \tanh^2 z)$	second order: u_{xx}, u_{tt}

- **the order of the equation** — sets the smoothness
- **tanh** — the activation of every Part 2 network
- **sine** — one exception, for waves, in L7.2

TANH AND ITS FIRST TWO DERIVATIVES



All three are smooth. Amber is tanh's second derivative: the curve a Laplacian in the residual sees.

Operator Form

Operator form — the equation written as $\mathcal{F}[u] = 0$ in the domain Ω: the differential operator $\mathcal{F}$ is what the equation does to the true field u, with physical parameters c. The residual $r = \mathcal{F}[\mathcal{N}]$ is the same operator applied to the network instead; training drives it to zero.

ONE EQUATION FOR EVERY PROBLEM IN PART 2

$$\mathcal{F}(u;\ \mathbf{x}, t, c) = 0, \quad \mathbf{x} \in \Omega,\ t \in [0, t_{\text{end}}]$$

equation $\mathcal{F}[u] = u_{xx} + u_{yy} + f(x, y)$

$\mathcal{F}$ the operator, c its parameters, $f(x, y)$ the source

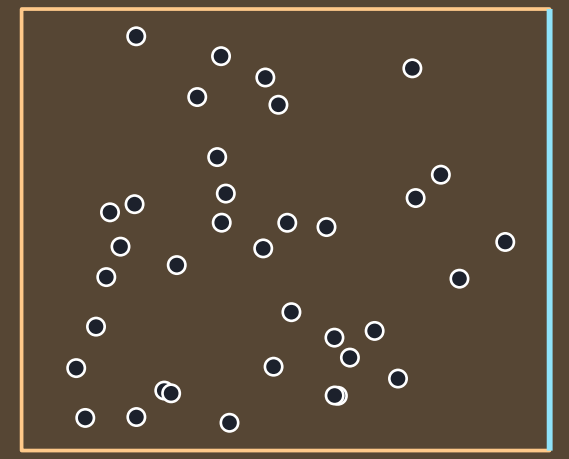
residual $r(\mathbf{x}) = \mathcal{F}[\mathcal{N}_\theta](\mathbf{x})$

r the residual : the network in place of u, zero where it solves

THE RESIDUAL IN PYTORCH

```
xy.requires_grad_(True); u = net(xy)          # the network at the points
du = torch.autograd.grad(u.sum(), xy, create_graph=True)[0]   # ∇u
u_xx = torch.autograd.grad(du[:, 0].sum(), xy, create_graph=True)[0][:, 0]
r = u_xx + u_yy + f                # the residual; u_yy likewise
```

THE DOMAIN AND ITS BOUNDARY



n is the outward normal: $\partial u/\partial n$ is the change across the wall, the flux. $\bar{u}$ is a given value, $\bar{q}$ a given flux; $\bar{q} = 0$ is an insulated wall.

Boundary and Initial Conditions

Boundary and initial conditions — the extra equations that make a problem well posed: a Dirichlet condition fixes the value on part of the boundary, a Neumann condition its normal derivative, an initial condition the field at $t = 0$, and an initial velocity its rate at $t = 0$ when the equation is second order in time.

FOUR CONDITIONS, FOUR RESIDUALS

DBC $u(\mathbf{x}, t) = \bar{u}(\mathbf{x}, t) \quad \forall\, \mathbf{x} \in \partial\Omega_{DBC}$

NBC $\mathcal{B}(\mathbf{x}, t, c) = 0 \quad \forall\, \mathbf{x} \in \partial\Omega_{NBC}$

IC $\mathcal{I}(\mathbf{x}, 0, c) = 0 \quad \forall\, \mathbf{x} \in \Omega$

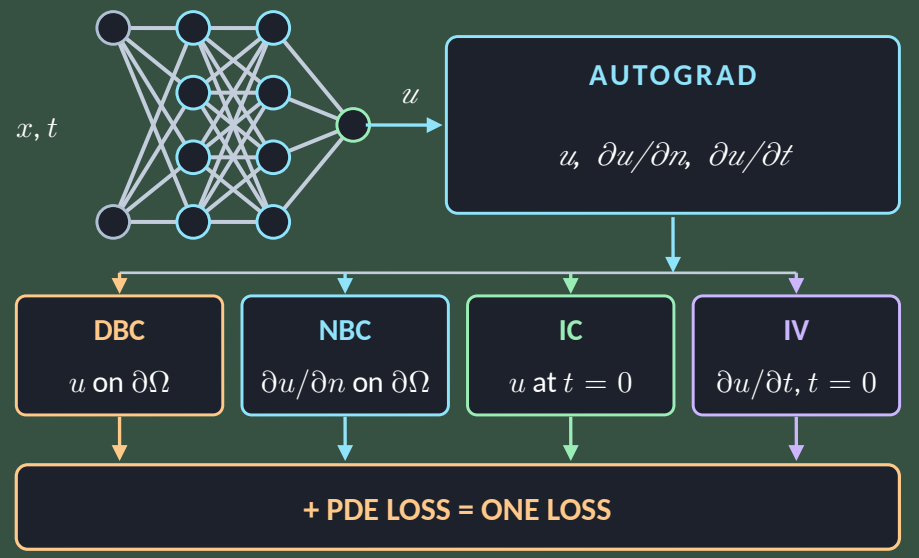
IV $\mathcal{V}(\mathbf{x}, 0, c) = 0 \quad \forall\, \mathbf{x} \in \Omega$

$\bar{u}$ the wall value, $\mathcal{B}$ the flux condition, $\mathcal{I}, \mathcal{V}$ the initial field and velocity

THE CONDITIONS IN PYTORCH

```
B_d = net(x_wall) - u_bar                  # Dirichlet: the value
B_n = (grad(u, x_flux) * n).sum(1) - q_bar   # Neumann: the flux ∂u/∂n
I = net(x_t0) - u_0                        # initial: u at t = 0
V = grad(u, x_t0)[:, -1:] - v_0            # velocity: ∂u/∂t at t = 0
```

WHERE EACH CONDITION ENTERS



a gradient step on the weights, then again

Poisson: walls only · heat: walls and IC · wave: walls, IC and IV.

Forward and Inverse Problems

Forward and inverse problems — in a forward problem the equation and its parameters are known and the network finds the field. In an inverse problem some parameters c are unknown: they become trainable beside the weights, and sparse measurements, added to the loss as a data term, identify them.

TWO SETTINGS, ONE LOOP

$$u_t + \mathcal{F}[u; c] = 0$$

$\mathcal{F}$ the operator, c its parameters; Raissi writes $\mathcal{N}[u; \lambda]$

$$\mathcal{L}(\theta, c) = \overline{r(\mathbf{x}; c)^2} + w_d \, \overline{(\mathcal{N}_\theta(\mathbf{x}_s) - u_s)^2}$$

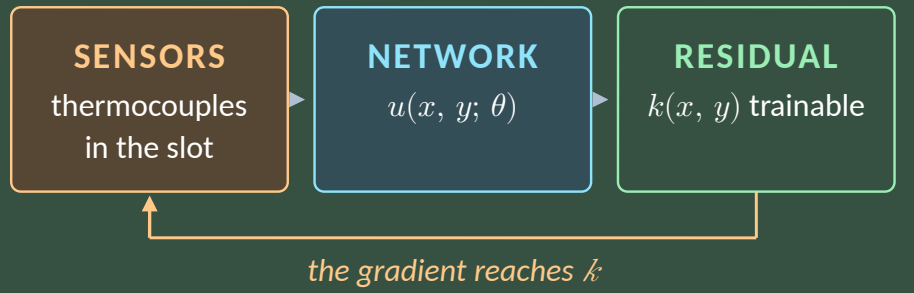
r the residual with c inside, u_s the readings at the sensors $\mathbf{x}_s$, w_d their weight

AN INVERSE PROBLEM IN PYTORCH

```
c = nn.Parameter(torch.tensor(1.0))                # c, the unknown parameter
opt = torch.optim.Adam([*net.parameters(), c])      # trained beside θ
res = r(net, x, c).pow(2).mean()                    # residual, c inside
loss = res + w_d * (net(x_s) - u_s).pow(2).mean()   # + data at the sensors
```

THE TWO SETTINGS

	forward	inverse
given	$\mathcal{F}$, c, conditions	$\mathcal{F}$, u at a few sensors
unknown	u	u and c
data term	absent	essential
cost	one run per c	one run
example	new slot: k known	aged slot: find k



One run: the temperature and the aged insulation.

Composite Loss

Composite loss — the sum of one mean squared residual per equation: the PDE on interior points, each condition on its own boundary or initial points, and the measurements if there are any, each multiplied by a weight. Every term vanishes at the exact solution.

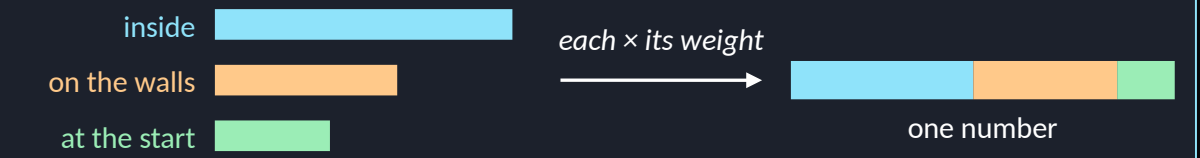
SIX TERMS, EACH A MEAN SQUARED RESIDUAL

$$\mathcal{L} = \mathcal{L}_{data} + w_{PDE}\mathcal{L}_{PDE} + w_{DBC}\mathcal{L}_{DBC} + w_{NBC}\mathcal{L}_{NBC} + w_{IC}\mathcal{L}_{IC} + w_{IV}\mathcal{L}_{IV}$$

$$\mathcal{L}_{PDE} = \frac{1}{n_{PDE}} \sum_{i=1}^{n_{PDE}} [\mathcal{F}_\theta(\mathbf{x}_i, t_i)]^2$$

w the weight of each term, n_{PDE} the interior points, $\mathcal{F}_\theta$ the residual there

SEVERAL RESIDUALS, ONE NUMBER



- **every term** — zero at the exact solution

WHERE EACH TERM IS EVALUATED

$\mathcal{L}_{\text{data}}$	measurements, if any
$\mathcal{L}_{\text{PDE}}$	interior collocation points
$\mathcal{L}_{\text{DBC}}$	values on the boundary
$\mathcal{L}_{\text{NBC}}$	derivatives on the boundary
$\mathcal{L}_{\text{IC}}$	the field at $t = 0$
$\mathcal{L}_{\text{IV}}$	its rate at $t = 0$

Soft Enforcement

Soft enforcement — a condition imposed as a penalty term in the loss: the network is asked to meet it at sampled boundary or initial points, competing with the PDE term for the same weights, so it holds only approximately. Neumann and initial conditions are added the same way.

THE CONDITION AS A PENALTY

$$\mathcal{L}_{DBC} = \frac{1}{n_{DBC}} \sum_{i=1}^{n_{DBC}} [u_\theta(\mathbf{x}_i, t_i) - \bar{u}(\mathbf{x}_i, t_i)]^2 \qquad \mathcal{L}_{NBC} = \frac{1}{n_{NBC}} \sum_{i=1}^{n_{NBC}} [\partial_n u_\theta(\mathbf{x}_i, t_i) - \bar{q}(\mathbf{x}_i, t_i)]^2$$

$\bar{u}$ the given value, $\bar{q}$ the given flux, ∂_n the normal derivative, n the boundary points

- **it competes** — with the PDE term, for the same weights
- **met approximately** — its loss weight sets how closely

PULLED TOWARDS THE WALL VALUE, NOT FIXED TO IT

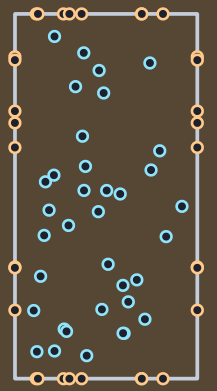


a small weight: a wide miss

a large weight: a narrow miss

THE SAME WALL, ASKED OR BUILT IN

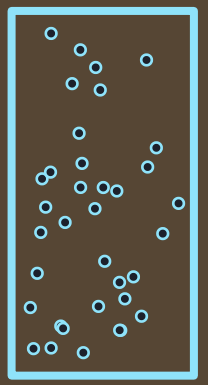
SOFT



loss + $w \cdot (u - \bar{u})^2$

at the wall: $u \approx \bar{u}$

HARD



$u = A + D \cdot N$

at the wall: $D = 0,\ u = \bar{u}$

Soft asks for the wall at the amber points and meets it nearly. Hard multiplies the network by D, zero on every wall, and meets it exactly.

Hard Enforcement

Hard enforcement — a plain network is $u = \mathcal{N}(x;\,\theta)$; hard enforcement writes the trial solution $u = A(x) + D(x)\cdot\mathcal{N}(x;\,\theta)$. A has the wall value and D is zero on the wall, so every θ meets it exactly, and training picks the best candidate. The price: a new D per geometry.

THE CONDITION BUILT INTO THE NETWORK

$$u_{\text{trial}} = A + D\,\mathcal{N}_\theta, \quad D(x,y) = x(1-x)\,y(1-y), \quad A = 0$$

D the mask : zero on the edges $x = 0, 1$ and $y = 0, 1$, positive inside; A the wall value

- D — a mask: zero on the wall, free inside
- **the boundary term** — gone, nothing left to balance
- **the rule** — soft by default, hard when D is easy

TALK · 2 MIN How would you explain the difference between a soft and a hard condition, in your own words? Then, in the table on the right, why is the hard network more accurate inside the square as well as on its walls?

MEASURED · POISSON, UNIT SQUARE

	soft, $w = 1$	soft, $w = 100$	hard
total error $\|u_\theta - u\| \,/\, \|u\|$	9.9e-3	7.0e-4	3.2e-5
wall error $\max\,\|u_\theta\|$	2.2e-2	1.5e-3	0
PDE residual, mean r^2	3.8e-4	7.2e-4	6.5e-5

Exact $u = \sin \pi x \cdot \sin \pi y$. Total error: the miss over the square relative to u (1e-2 = 1 %); wall error: the largest miss on the walls, where $u = 0$; PDE residual: the mean of r^2 inside. 101 × 101 test grid; tanh 40 x 2, 320 interior, 240 boundary, seed 0.

FROM WALL TO WALL



Loss Weight Rebalance

Loss weight rebalance — choosing the weights w_k in $\mathcal{L} = \mathcal{L}_{PDE} + \Sigma\, w_k\, \mathcal{L}_k$ so that every term pulls on θ as hard as the equation does. A term with a small gradient is neglected by the optimiser, and its condition is met only roughly; rebalancing raises its weight until it counts.

BY HAND: THREE GUIDELINES

- **scale first** — x, u and f of order one
- **start at** $w_k = 1$ — raise the term that stalls
- **sweep 1, 10, 100** — keep the lowest test error

AUTOMATIC: FROM GRADIENT NORMS

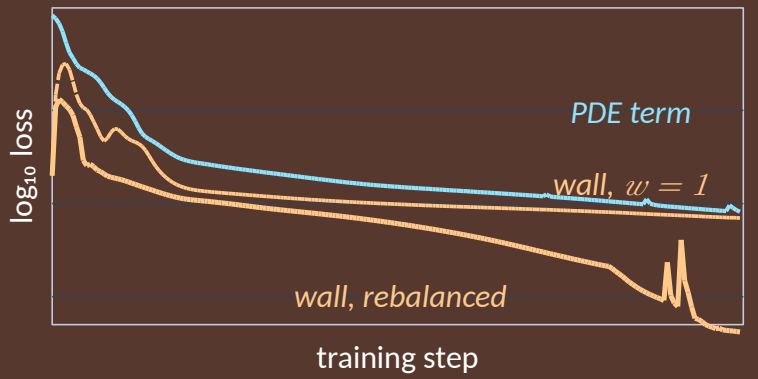
$$\hat{w}_k = \frac{\|\nabla_\theta \mathcal{L}_{PDE}\|}{\|\nabla_\theta \mathcal{L}_k\|} \qquad w_k \leftarrow 0.9\, w_k + 0.1\, \hat{w}_k$$

$\|\nabla_\theta \mathcal{L}_k\|$ how hard term k pulls on θ; $\hat{w}_k$ the new weight, w_k its running average

REBALANCING IN PYTORCH

```
g_pde = grad(L_pde, theta, retain_graph=True)  # pull of the PDE term
g_dbc = grad(L_dbc, theta, retain_graph=True)  # pull of the wall term
w_dbc = 0.9*w_dbc + 0.1*norm(g_pde)/norm(g_dbc)  # raise the wall
```

MEASURED · LOSS PER TERM



	$w = 1$	$w = 10$	$w = 100$	rebalanced
total error	9.9e-3	1.6e-3	7.0e-4	1.3e-3
wall error	2.2e-2	3.6e-3	1.5e-3	3.5e-3

At $w = 1$ the wall loss stalls; rebalanced, w rose to about 10 and the wall loss kept falling, near the best hand weight without a sweep.

Collocation

Collocation — evaluating the residual at a finite set of sampled points, the collocation points, in the domain and on its boundary. Between the points nothing is imposed, so their number and placement set the accuracy; the interior needs at least N^* of them.

HOW MANY POINTS: AT LEAST N*

course : $N^* = (D_i + 1) + (D + 1) \times K$

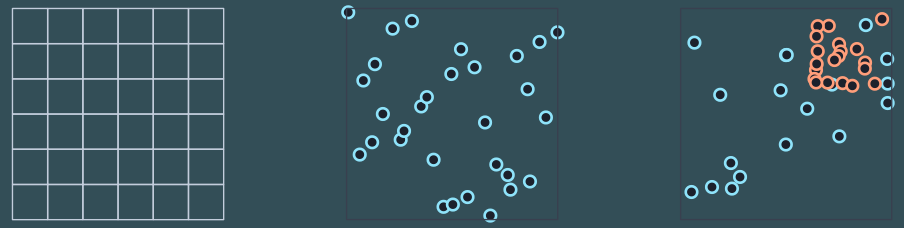
D_i inputs, K hidden layers of D neurons, each $+1$ a bias; Liu's p, N_L, N_n are the same

Liu : $P^* = (p + 1) + (N_n + 1) \times N_L$

- N^* — one condition per neuron and bias, not per weight
- **in practice** — 10 to 20 × N^*, Ex_07.1 uses 15

	FDM node	collocation point
carries	its own unknown value	no unknown: the weights are shared
how many	error ∝ h^2: refine to the accuracy	at least N^*, counted from the network
grows with	the mesh: $1 \,/\, h^2$	D neurons, K layers, D_i inputs
Ex_07.1	41 × 81 = 3 081 nodes, one current	2 000 = 15 N^*, all currents

THREE WAYS TO PLACE THE POINTS



uniform grid | Latin hypercube | adaptive

N*, POINTS AND FDM NODES · 3 INPUTS

network	N^*	points, 15 N^*	weights	FDM, as many
32 × 4	136	2 040	3 329	41 × 81
100 × 2	206	3 090	10 601	73 × 145
200 × 4	808	12 120	121 601	247 × 493

32 × 4: $N^* = (3 + 1) + (32 + 1) \times 4 = 136$. Points grow with the neurons, weights with their square. Orange: points added by refinement.

Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
energy method	an energy, first derivatives only	Liu ch. 2.8	elasticity, structures
the stencil	finite differences on the network	Liu ch. 2.7.6	costly derivatives
learned boundary	a second network for curved walls	Liu ch. 3.1 – 3.3	holes, curved geometry
adaptive sampling	points where the residual is large	Liu ch. 5.6	shocks, thin layers
periodic activations	sine networks for oscillations	Liu ch. 5.7.4	waves, vibration

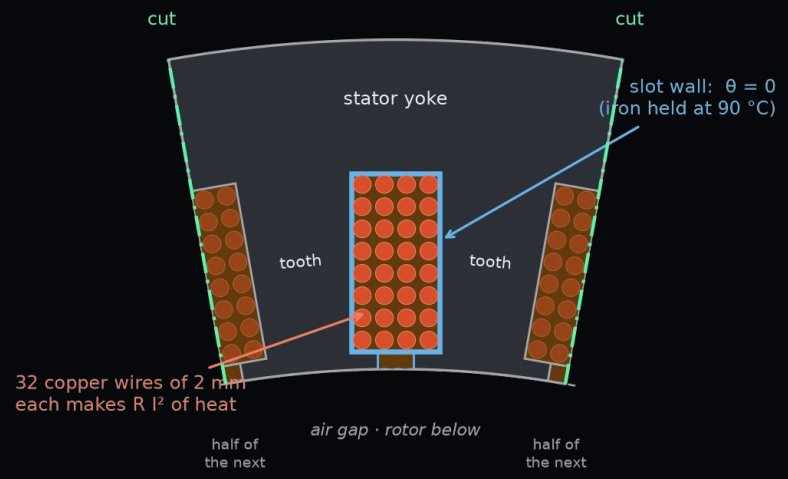
WHY THEY ARE LEFT OUT

Each answers a difficulty the problems of this block do not meet. A plain PINN, with soft or hard conditions, solves Ex_07.1; the rest return when a problem asks for them.

Exercise 7.1

ONE NOTEBOOK, SECTION BY SECTION

- **1** — the slot and the heat its winding makes
- **2** — the exact and the finite-difference solution
- **3** — the mesh density for an agreed 0.01 K
- **4 – 5** — a PINN, soft and hard, and the three compared
- **6 – 7** — the report; mini projects MP7.1A and MP7.1B



THE QUESTION EACH PART ANSWERS

1 – 2	how hot is it, exactly?
3	which mesh is fine enough?
4 – 5	soft or hard, and against FDM?

REPORT QUESTION, TO CONCLUDE

Finite differences met 0.01 K for almost nothing. For which problem would you train a physics-informed network instead, and what would you check it against?

SUMMARY

Key Takeaways

PINN	
what it is	a network that is the solution field u(x, t)
function	coordinates in, u out; derivatives by autograd
train, tune	tanh, at least N* points, Adam then L-BFGS
vs FEM	weights, not nodal values; no mesh
used for	forward and inverse problems, all of Part 2

PINN LOSS	
what it is	one mean squared residual per equation
function	soft: each condition asked at its own points
train, tune	weights as rates of exchange; balance them
vs soft	hard: exact on the wall, and more accurate
used for	every block of Part 2, a different residual

Next — L7.2: the fundamental equations, and the conditions each one needs.

Questions

1 – 2 · THE NETWORK AS THE SOLUTION

1 **How can a network be trained to be the temperature field in the slot without any solved data?**

The network takes the position in the slot and the current as inputs and returns the temperature rise there. Automatic differentiation gives its derivatives with respect to the inputs, and the loss is the residual of the steady conduction equation at points inside the slot, plus the wall conditions; no solution is needed as a label. With the current as an input, one training answers every current, and an unknown parameter can be trained beside the weights. *Liu ch. 2.1 – 2.2; Ex_07.1 section 4*

2 **How does a network differ from finite differences as a way to solve the slot, and when does it pay?**

Finite differences solve for the temperature at the nodes of a mesh, whose spacing sets the accuracy. A network solves for weights shared by every point, with exact derivatives by automatic differentiation and sampled points in place of the mesh. In Ex_07.1 finite differences met the agreed 0.01 K in milliseconds, which neither network did. The network pays when one training answers many currents, when a mesh is costly, or when a parameter is found from data. *Liu ch. 1 – 2; Ex_07.1 section 5*

3 – 4 · THE LOSS AND ITS POINTS

3 **How is a wall temperature imposed on a network softly and by hard enforcement, and why is the hard one more accurate?**

Soft enforcement adds the wall condition to the loss at points on the walls, where it competes with the equation and holds only approximately; its error spreads into the slot. Hard enforcement multiplies the network by a mask that is zero on every wall, so the condition holds for any weights. In Ex_07.1 the hard network was within 0.02 K everywhere, the soft one about 1.5 K off near the walls. The price is a new mask for each geometry. *Liu ch. 2.5.3; Ex_07.1 sections 4 – 5*

4 **How are the terms of a network's composite loss balanced, and how many collocation points does the network need?**

Each equation and condition adds one mean squared residual, and the terms differ in units and size; a term with a small gradient is neglected by the optimiser. The variables are first scaled to order one, then a weight is raised by hand or from the ratio of the gradient norms during training. The residual is imposed only at the collocation points, so their number sets the accuracy: at least one per neuron and bias, in practice ten to twenty times that. *Liu ch. 2.2.3, 2.6; Ex_07.1 section 4*

NEXT

L7.2 — Fundamental PDEs

The same loop for every equation: only the residual and its conditions change.

BEFORE THE NEXT SESSION

read	Liu ch. 2.1 – 2.9
run	Ex_07.1, sections 1 to 5
bring	the wall error, soft and hard
think about	which conditions your problem carries

Part 2 has one method and many applications. This was the method.